

ONEDRAFT

CAD-AI — Aria Powered

Runtime & Application Orchestration Stack

Version 0.1

Status: Implementation Specification

Architecture: Service-Oriented Modular Monolith → Worker-Based Execution

Runtime: Python

API: REST / JSON

Database: PostgreSQL

Queue: Redis-backed job system

Workers: Python asynchronous workers

AI Layer: Aria Orchestration

Geometry Layer: Geometry Service

Compliance Layer: Regulatory Engine

Documentation Layer: Drawing / Document Engine

API Version: /api/v1

1. RUNTIME DESIGN PRINCIPLE

The OneDraft runtime is the execution layer between the external API and the persistent project model.

The runtime is responsible for transforming:

user intent

into:

validated domain operations

and ultimately:

versioned project state and derived documentation.

The runtime therefore becomes the computational control plane of OneDraft.

The fundamental execution path is:

```
CLIENT
↓
API GATEWAY
↓
APPLICATION SERVICE
```

↓
DOMAIN MODEL
↓
COMMAND BOUNDARY
↓
TRANSACTION
↓
JOB DISPATCH
↓
WORKER
↓
SPECIALIZED ENGINE
↓
VALIDATION
↓
MODEL FINALIZATION
↓
DOCUMENTATION

2. RUNTIME ARCHITECTURE

The initial runtime shall be implemented as a modular application rather than prematurely decomposed into independent microservices.

The initial architecture is:

```
apps/  
├── api/  
│   ├── routes  
│   ├── middleware  
│   ├── dependencies  
│   └── application interface  
├── worker/  
│   ├── jobs  
│   ├── queues  
│   └── task execution  
└── web/  
    └── frontend
```

The domain and reusable application logic reside in:

```
packages/  
├── domain/  
├── application/  
├── infrastructure/  
├── geometry/  
├── compliance/  
├── documentation/  
└── ai/
```

This prevents business logic from becoming trapped inside HTTP handlers.

3. APPLICATION LAYER

The application layer coordinates use cases.

It does not own raw database implementation.

It does not perform HTTP-specific logic.

It does not directly implement geometry algorithms.

It does not directly call an external AI provider.

Its responsibility is orchestration.

Examples:

CreateProject
CreateRequirement
CreateBuilding
CreateLevel
CreateSpace
CreateElement
ExecuteAriaCommand
CreateRevision
CreateRedline
ResolveRedline
RunValidation
GenerateDrawings
GenerateDocumentPackage
RecordAcceptance

4. DOMAIN LAYER

The domain layer represents the authoritative OneDraft computational model.

Core domain objects include:

Project
Requirement
Site
Building
Level
Grid
Space
Element
Assembly
Constraint
Relationship
ModelVersion
Revision
ChangeRecord
AriaCommand
DesignProposal
View

Drawing
Sheet
Redline
ValidationResult
DocumentPackage
AcceptanceRecord

The domain layer must remain independent of:

HTTP
PostgreSQL
Redis
AI providers
file storage
cloud infrastructure
frontend frameworks

5. DOMAIN COMMAND BOUNDARY

All model mutations must enter through an explicit command boundary.

Conceptually:

```
Command
  ↓
Authorization
  ↓
Revision Check
  ↓
Constraint Check
  ↓
Domain Validation
  ↓
Mutation
  ↓
Change Record
  ↓
Revision
  ↓
Event
```

No application component may silently mutate model state outside this boundary.

6. COMMAND INTERFACE

The primary interface shall be:

```
ProjectModelService.execute(command)
```

The command contains:

project_id
revision_id
operation
target_objects
parameters
preserve_constraints
requested_by
expected_revision

The service returns:

command_id
status
revision_id
model_version_id
affected_objects
changes
events
jobs

7. COMMAND TYPES

Initial command operations:

CREATE_PROJECT
UPDATE_PROJECT

CREATE_REQUIREMENT
UPDATE_REQUIREMENT

CREATE_BUILDING
UPDATE_BUILDING

CREATE_LEVEL
UPDATE_LEVEL

CREATE_SPACE
UPDATE_SPACE

CREATE_ELEMENT
UPDATE_ELEMENT
DELETE_ELEMENT

MOVE_ELEMENT
ROTATE_ELEMENT
RESIZE_ELEMENT
CHANGE_ELEMENT_PARAMETER

CREATE_ASSEMBLY
UPDATE_ASSEMBLY

CREATE_CONSTRAINT
UPDATE_CONSTRAINT

CREATE_RELATIONSHIP

DELETE_RELATIONSHIP

CREATE_VIEW
UPDATE_VIEW

CREATE_REDLINE
RESOLVE_REDLINE
REJECT_REDLINE

CREATE_DESIGN_PROPOSAL
APPROVE_DESIGN_PROPOSAL
REJECT_DESIGN_PROPOSAL

8. COMMAND LIFECYCLE

Every command follows:

RECEIVED
↓
AUTHORIZING
↓
VALIDATING
↓
PROPOSED
↓
EXECUTING
↓
COMMITTING
↓
COMPLETED

Failure states:

REJECTED
FAILED
CANCELLED
STALE
CONFLICT

A command may not transition directly from:

RECEIVED

to:

COMPLETED

without validation.

9. REVISION CONCURRENCY

Every mutating request contains an expected revision.

Example:

```
expected_revision = 17
```

The service compares this against the project's current revision.

If:

```
current_revision != expected_revision
```

the command becomes:

```
STALE
```

The system does not silently overwrite the newer project state.

This provides optimistic concurrency control.

10. TRANSACTION SERVICE

The transaction service is responsible for atomic project-state changes.

Conceptually:

```
BEGIN
↓
Load project
↓
Load current revision
↓
Verify expected revision
↓
Authorize actor
↓
Load affected objects
↓
Validate command
↓
Validate constraints
↓
Apply mutation
↓
Create model version
↓
Create revision
↓
Create change records
↓
Create event
↓
```

COMMIT

Any failure causes:

ROLLBACK

11. DOMAIN VALIDATION

Domain validation occurs before persistence.

Examples:

level elevation cannot conflict with project level ordering
space cannot have invalid boundary state
element host must exist
relationship endpoints must exist
element cannot reference nonexistent assembly
constraint target must exist
revision must belong to project
command target must belong to project

Domain validation is independent from building-code validation.

12. REGULATORY VALIDATION SEPARATION

OneDraft shall maintain a strict distinction between:

domain validity

and:

regulatory compliance.

For example:

A wall may be geometrically valid

while:

the proposed wall assembly may fail a code requirement.

Therefore:

DOMAIN VALIDATION

and:

are separate execution stages.

13. ARIA ORCHESTRATION

Aria is the intelligence layer responsible for translating natural-language project instructions into structured OneDraft operations.

The execution chain becomes:

```
USER
↓
ARIA
↓
INTENT INTERPRETATION
↓
TARGET IDENTIFICATION
↓
COMMAND PROPOSAL
↓
CONSTRAINT ANALYSIS
↓
DOMAIN VALIDATION
↓
COMMAND EXECUTION
```

Aria does not directly modify PostgreSQL.

14. ARIA PROVIDER ADAPTER

The AI infrastructure shall use an adapter boundary:

```
AriaOrchestrator
↓
AIProviderAdapter
↓
Model Provider
```

The domain application does not depend on the identity of the underlying model provider.

This preserves provider independence.

15. ARIA REQUEST PIPELINE

An Aria request proceeds through:

NATURAL LANGUAGE
↓
CONTEXT ASSEMBLY
↓
MODEL INTERPRETATION
↓
STRUCTURED INTENT
↓
TARGET RESOLUTION
↓
COMMAND GENERATION
↓
COMMAND VALIDATION
↓
EXECUTION

The AI response itself is not considered authoritative project state.

Only the validated command result becomes project state.

16. ARIA CONTEXT

Aria receives only the project context required for the operation.

Context may include:

- project metadata
- current revision
- model version
- affected objects
- relationships
- constraints
- requirements
- relevant code manifest
- previous redlines
- drawing context
- validation results

The entire database must not be indiscriminately transmitted to the model.

17. ARIA OUTPUT CONTRACT

Aria should produce structured output.

Conceptually:

```
{  
  "intent": "MOVE_ELEMENT",  
  "targets": [  
    "uuid"  
  ],  
}
```

```
{
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  },
  "preserve_constraints": [
    "uuid"
  ],
  "confidence": 0.96
}
```

The application layer converts this into a domain command.

18. AI CONFIDENCE

AI confidence is informational.

Confidence alone must never authorize an unsafe model mutation.

The authoritative sequence remains:

```
AI INTERPRETATION
↓
DOMAIN VALIDATION
↓
CONSTRAINT VALIDATION
↓
EXECUTION
```

19. DESIGN PROPOSAL MODE

For complex changes, Aria may produce a design proposal rather than immediately executing a command.

Example:

```
USER REQUEST
↓
ARIA
↓
DESIGN PROPOSAL
↓
IMPACT ANALYSIS
↓
USER / SYSTEM APPROVAL
↓
COMMAND
↓
EXECUTION
```

This is required where multiple dependent objects may be affected.

20. IMPACT ANALYSIS

Before executing a complex command, the system determines:

- affected elements
- affected spaces
- affected levels
- affected relationships
- affected constraints
- affected views
- affected drawings
- affected schedules
- affected validation rules

The result becomes an impact graph.

21. DEPENDENCY GRAPH

The existing:

`model.relationships`

table forms the persistence layer for dependency relationships.

The runtime builds an execution graph:

```
OBJECT
↓
DEPENDENCIES
↓
DERIVED OBJECTS
↓
DOCUMENTATION
↓
VALIDATION
```

This allows OneDraft to determine which derived artifacts require regeneration.

22. DIRTY STATE

Objects affected by a model change become dirty.

Conceptually:

CLEAN

DIRTY
REGENERATING
VALID
INVALID

A wall modification may mark:

wall
space
plan view
section view
elevation
dimension
schedule
validation result
drawing
document package

as requiring regeneration or revalidation.

23. JOB SYSTEM

Long-running operations are dispatched to the worker system.

Initial job types:

GEOMETRY_GENERATION
GEOMETRY_VALIDATION
ARIA_PROCESSING
COMPLIANCE_VALIDATION
DRAWING_GENERATION
SCHEDULE_GENERATION
DOCUMENT_GENERATION
DOCUMENT_HASHING
EXPORT
REBUILD_DERIVED_STATE

24. QUEUE ARCHITECTURE

The initial queue topology is:

oneDraft.default
oneDraft.aria
oneDraft.geometry
oneDraft.validation
oneDraft.documentation
oneDraft.export

The queues may initially be implemented through Redis-backed task infrastructure.

25. JOB ENVELOPE

Every job contains:

job_id
project_id
revision_id
model_version_id
job_type
priority
attempt
payload
created_at
started_at
completed_at

The model version is mandatory for any job operating on project geometry or documentation.

26. JOB IDEMPOTENCY

Jobs must be safely retryable.

A job execution must not create duplicate:

revisions
drawings
document packages
validation runs
events

when retried.

The job system therefore uses:

job_id
idempotency_key
source_revision
source_model_version

to identify execution state.

27. JOB FAILURE

Failed jobs transition to:

FAILED

with:

error_code
error_message
retryable
attempt_count
failure_context

Transient infrastructure failures may be retried.

Deterministic domain failures must not be blindly retried.

28. DEAD-LETTER PROCESSING

Jobs exceeding their retry policy enter:

DEAD_LETTER

The dead-letter record preserves:

original_job_id
project_id
revision_id
model_version_id
failure_history
payload_hash

This prevents silent disappearance of failed processing.

29. WORKER EXECUTION MODEL

Workers execute specialized jobs.

Conceptually:

```
Worker
↓
Load Job
↓
Verify Model Version
↓
Load Required State
↓
Execute Engine
↓
Validate Output
↓
Persist Derived State
↓
Emit Event
↓
Complete Job
```

30. STALE JOB PROTECTION

Before committing results, a worker verifies:

```
project_id  
revision_id  
model_version_id
```

remain valid.

If the project has advanced beyond the job's source model:

STALE

The worker must not silently promote the stale result to current state.

31. GEOMETRY ENGINE BOUNDARY

The geometry engine is responsible for computational geometry.

It receives:

```
model version  
elements  
assemblies  
parameters  
relationships  
constraints
```

and produces:

```
geometry  
bounding volumes  
topology  
intersections  
derived measurements
```

The geometry engine does not own project truth.

32. GEOMETRY OUTPUT

Geometry output is stored through:

```
geometry.geometry_references
```

and external/object storage where appropriate.

Every output is associated with:

project
model version
object
geometry version
geometry hash

33. GEOMETRY INVALIDATION

Geometry becomes invalid when its source model state changes.

The runtime determines:

`source_model_version != current_required_model_version`

and marks the geometry stale.

34. COMPLIANCE ENGINE

The compliance engine consumes:

project
model version
requirements
jurisdiction
code manifest
code rules

and produces:

validation runs
validation results

The compliance engine must identify the regulatory source behind every material result.

35. VALIDATION PIPELINE

The complete validation sequence becomes:

```
MODEL
↓
DOMAIN VALIDATION
↓
GEOMETRY VALIDATION
↓
CODE VALIDATION
```

↓
ACCESSIBILITY VALIDATION
↓
COORDINATION VALIDATION
↓
DOCUMENT VALIDATION
↓
VALIDATION SUMMARY

36. DOCUMENTATION ENGINE

The documentation engine converts the computational project model into architectural documentation.

Its inputs include:

model version
revision
views
drawing definitions
sheet templates
annotations
dimensions
schedules
graphic standards

Its outputs include:

drawings
sheets
PDF
other supported document formats

37. DRAWING DERIVATION

Drawings are derived from model state.

The relationship is:

MODEL
↓
VIEW
↓
DRAWING
↓
SHEET
↓
DOCUMENT PACKAGE

A drawing must never become an independent substitute for the underlying model.

38. DOCUMENTATION REGENERATION

When a model modification affects documentation:

```
MODEL CHANGE
↓
IMPACT ANALYSIS
↓
DIRTY VIEWS
↓
REGENERATE DRAWINGS
↓
REVALIDATE
↓
REBUILD PACKAGE
```

Unaffected documentation may remain unchanged where provenance permits.

39. REDLINE EXECUTION

A redline is treated as an instruction against project state.

The lifecycle is:

```
OPEN
↓
INTERPRETING
↓
PROPOSED
↓
IMPLEMENTING
↓
VALIDATING
↓
RESOLVED
```

A redline cannot be marked resolved merely because Aria generated a response.

The underlying project modification must be completed and validated.

40. REDLINE IMPACT

Every redline resolution records:

```
redline_id
command_id
source_revision
result_revision
implemented_changes
validation_results
```

This creates a traceable chain from human markup to model modification.

41. EVENT BUS

The runtime emits domain events after successful state transitions.

Initial events include:

PROJECT_CREATED
REQUIREMENT_CREATED
MODEL_CHANGED
REVISION_CREATED
COMMAND_COMPLETED
REDLINE_CREATED
REDLINE_RESOLVED
VALIDATION_STARTED
VALIDATION_COMPLETED
DRAWING_GENERATED
DOCUMENT_PACKAGE_GENERATED
ACCEPTANCE_RECORDED

42. EVENT PRINCIPLE

Events describe what happened.

Commands describe what should happen.

Therefore:

COMMAND

is an instruction.

EVENT

is an immutable historical fact within the application record.

The two must not be conflated.

43. EVENT PAYLOAD

An event contains:

event_id
event_type
project_id
revision_id

object_id
actor_id
timestamp
payload
schema_version

The payload must contain sufficient information for downstream consumers without exposing unnecessary private data.

44. AUDIT PIPELINE

Every material project mutation produces an audit event.

The chain becomes:

```
COMMAND
↓
DOMAIN MUTATION
↓
REVISION
↓
CHANGE RECORD
↓
AUDIT EVENT
```

This permits reconstruction of how the current state was produced.

45. AUTHORIZATION PIPELINE

Every protected request passes through:

```
TOKEN
↓
IDENTITY
↓
ORGANIZATION MEMBERSHIP
↓
PROJECT ACCESS
↓
API SCOPE
↓
RESOURCE AUTHORIZATION
↓
APPLICATION SERVICE
```

Authorization is not delegated to the frontend.

46. PROJECT TENANCY

Every project belongs to exactly one organization.

Runtime queries must be project-scoped.

The application must never retrieve project objects using only an object UUID without also verifying project ownership or membership.

47. SERVICE CONTEXT

Each application request receives a service context containing:

```
request_id
actor_id
organization_id
project_id
authorization_scopes
correlation_id
```

This context propagates through synchronous and asynchronous operations.

48. CORRELATION ID

Every API request receives:

```
request_id
```

Jobs spawned by that request retain:

```
correlation_id
```

This allows tracing:

```
USER REQUEST
↓
API
↓
COMMAND
↓
JOB
↓
WORKER
↓
ENGINE
↓
DATABASE
```

49. OBSERVABILITY

The runtime shall produce:

- structured logs
- metrics
- traces
- job telemetry
- database timing
- AI execution telemetry

Logs must be machine-readable.

50. LOGGING PRINCIPLE

Logs must identify:

- request_id
- project_id
- job_id
- command_id
- revision_id
- model_version_id

where applicable.

Sensitive user content and credentials must not be indiscriminately logged.

51. HEALTH ENDPOINT

The API shall expose:

GET /health

for process-level health.

A separate readiness endpoint:

GET /ready

checks required runtime dependencies.

Conceptually:

- API
- POSTGRES
- REDIS
- WORKER
- OBJECT STORAGE

52. CONFIGURATION

Runtime configuration is supplied through environment configuration.

Examples:

```
DATABASE_URL
REDIS_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
AI_PROVIDER
AI_MODEL
JWT_ISSUER
JWT_AUDIENCE
LOG_LEVEL
ENVIRONMENT
```

Secrets must never be committed to source control.

53. ENVIRONMENT MODEL

Initial environments:

```
development
test
staging
production
```

Development and test environments must not share production credentials or production project data.

54. OBJECT STORAGE

Large artifacts should not be stored directly inside PostgreSQL.

Object storage is used for:

```
source documents
code artifacts
geometry payloads
rendered drawings
PDF packages
exports
generated files
```

PostgreSQL stores the authoritative metadata and references.

55. OBJECT STORAGE PROVENANCE

Every stored artifact receives:

object_id
project_id
revision_id
model_version_id
content_hash
media_type
size
storage_reference
created_at

The hash permits integrity verification.

56. ARTIFACT HASHING

Generated artifacts shall be hashed.

Initial algorithm:

SHA-256

The hash is recorded before the artifact is considered complete.

57. DOCUMENT PACKAGE FINALIZATION

A document package may become:

GENERATED

only after:

drawing generation
validation
artifact creation
artifact hashing
metadata persistence

have succeeded.

58. ACCEPTANCE BOUNDARY

Customer acceptance occurs only against a specific:

project
revision
model version
code manifest
document package

The acceptance record therefore becomes immutable historical evidence of which package was accepted.

59. API REQUEST PIPELINE

Every request follows:

```
HTTP
↓
REQUEST ID
↓
AUTHENTICATION
↓
AUTHORIZATION
↓
VALIDATION
↓
ROUTE
↓
APPLICATION SERVICE
↓
DOMAIN
↓
PERSISTENCE
↓
RESPONSE
```

60. API RESPONSE PIPELINE

Responses use the previously established envelope:

```
{
  "data": {},
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

Errors use the established error contract.

61. ASYNCHRONOUS API PATTERN

Expensive requests return:

202 ACCEPTED

with:

job_id
status
request_id

The client then uses:

GET /api/v1/jobs/{job_id}

to retrieve progress.

62. SYNCHRONOUS API PATTERN

Fast transactional operations may return immediately.

Examples:

create project
create requirement
create level
create space
create element
create constraint
create relationship

provided execution remains within acceptable transaction limits.

63. ASYNCHRONOUS OPERATIONS

The following should normally be asynchronous:

AI processing
complex geometry
code validation
drawing generation
document generation
large exports
package hashing
bulk regeneration

64. APPLICATION SERVICE REGISTRY

Initial services:

```
ProjectService
RequirementService
ModelService
CommandService
RevisionService
AriaService
RedlineService
ValidationService
DrawingService
DocumentService
AcceptanceService
JobService
AuditService
```

These are logical services within the application architecture.

They need not initially be separate deployed processes.

65. REPOSITORY INTERFACES

The domain/application layer communicates with persistence through repository interfaces.

Examples:

```
ProjectRepository
ElementRepository
RevisionRepository
CommandRepository
ConstraintRepository
RelationshipRepository
DrawingRepository
ValidationRepository
DocumentRepository
AuditRepository
```

Concrete PostgreSQL implementations reside in infrastructure.

66. UNIT OF WORK

The transaction boundary shall be represented through a Unit of Work abstraction.

Conceptually:

```
with unit_of_work:
    project = projects.get(project_id)
    command_service.execute(command)
```

```
unit_of_work.commit()
```

The Unit of Work owns transaction coordination.

67. DATABASE SESSION RULE

HTTP handlers must not directly manage arbitrary PostgreSQL transactions.

Application services receive the appropriate Unit of Work or transaction context.

This keeps persistence mechanics outside the API layer.

68. CACHE STRATEGY

Caching is permitted for:

```
read-heavy reference data
code metadata
sheet templates
static configuration
non-authoritative derived information
```

The cache must never become the authoritative source for project state.

69. PROJECT STATE AUTHORITY

The authoritative hierarchy remains:

```
POSTGRESQL PROJECT MODEL
      ↓
REVISION HISTORY
      ↓
DERIVED ARTIFACTS
```

Redis and caches are disposable execution infrastructure.

70. SECURITY BOUNDARY

The runtime must enforce:

```
authentication
authorization
tenant isolation
```

input validation
SQL parameterization
secret management
artifact access control
audit logging
rate limiting

No AI-generated content bypasses these controls.

71. AI SECURITY BOUNDARY

Aria-generated commands are treated as untrusted input until validated.

The pipeline is:

```
ARIA OUTPUT
↓
SCHEMA VALIDATION
↓
AUTHORIZATION
↓
DOMAIN VALIDATION
↓
CONSTRAINT VALIDATION
↓
EXECUTION
```

Natural-language instructions cannot directly become database operations.

72. FILE SECURITY

Uploaded project files must pass:

authentication
authorization
file type validation
size validation
malware/security scanning
content hashing
metadata registration

before entering project processing.

73. RATE LIMITING

Initial rate limits should distinguish:

normal API operations
AI operations
document generation
bulk operations
authentication
file uploads

AI and document generation receive stricter resource controls because of computational cost.

74. RESOURCE GOVERNANCE

Every asynchronous job has:

priority
timeout
retry policy
resource class

Initial resource classes:

LIGHT
STANDARD
HEAVY
CRITICAL

75. JOB PRIORITY

Default priority:

interactive model operation > validation > documentation > bulk regeneration

This prevents large background jobs from starving interactive design operations.

76. INTERACTIVE DESIGN LOOP

The primary user experience is:

USER
↓
ARIA
↓
PROPOSAL
↓
COMMAND
↓
MODEL UPDATE
↓

IMPACT ANALYSIS
↓
DRAWING UPDATE
↓
VALIDATION
↓
USER REVIEW

This is the core OneDraft computational loop.

77. REDLINE LOOP

The redline workflow becomes:

DRAWING
↓
USER REDLINE
↓
REDLINE RECORD
↓
ARIA INTERPRETATION
↓
COMMAND
↓
MODEL REVISION
↓
DERIVED DOCUMENT UPDATE
↓
VALIDATION
↓
REDLINE RESOLUTION

The drawing is therefore an interface to the model rather than an isolated image.

78. FULL PROJECT PIPELINE

The complete runtime pipeline becomes:

PROJECT
↓
REQUIREMENTS
↓
SITE
↓
BUILDING
↓
LEVELS
↓
SPACES
↓
ELEMENTS

↓
CONSTRAINTS
↓
RELATIONSHIPS
↓
ARIA
↓
COMMANDS
↓
MODEL VERSION
↓
REVISION
↓
GEOMETRY
↓
COMPLIANCE
↓
VIEWS
↓
DRAWINGS
↓
SHEETS
↓
DOCUMENT PACKAGE
↓
CUSTOMER REVIEW
↓
REDLINES
↓
REVISION
↓
REVALIDATION
↓
ACCEPTANCE

79. APPLICATION PACKAGE STRUCTURE

The initial Python package structure shall be:

```
packages/application/  
├── services/  
│   ├── project_service.py  
│   ├── requirement_service.py  
│   ├── model_service.py  
│   ├── command_service.py  
│   ├── revision_service.py  
│   ├── aria_service.py  
│   ├── redline_service.py  
│   ├── validation_service.py  
│   ├── drawing_service.py  
│   ├── document_service.py  
│   ├── acceptance_service.py  
│   ├── job_service.py  
│   └── audit_service.py
```

- commands/
- queries/
- dto/
- unit_of_work.py

80. INFRASTRUCTURE PACKAGE

```
packages/infrastructure/  
├── database/  
│   ├── session.py  
│   ├── repositories/  
│   └── migrations/  
├── queue/  
│   ├── client.py  
│   ├── jobs.py  
│   └── worker.py  
├── storage/  
│   ├── object_store.py  
│   └── artifacts.py  
├── auth/  
│   └── provider.py  
└── telemetry/  
    ├── logging.py  
    ├── metrics.py  
    └── tracing.py
```

81. AI PACKAGE

```
packages/ai/  
├── aria/  
│   ├── orchestrator.py  
│   ├── context.py  
│   ├── interpreter.py  
│   ├── planner.py  
│   ├── command_builder.py  
│   └── safety.py  
└── providers/  
    ├── base.py  
    └── adapters/
```

82. GEOMETRY PACKAGE

```
packages/geometry/  
├── model.py  
├── primitives.py  
├── topology.py  
├── intersections.py  
├── measurements.py  
├── validation.py  
└── engine.py
```

The geometry package remains independent of the API.

83. COMPLIANCE PACKAGE

```
packages/compliance/  
├── manifest.py  
├── sources.py  
├── rules.py  
├── applicability.py  
├── evaluator.py  
├── results.py  
└── engine.py
```

84. DOCUMENTATION PACKAGE

```
packages/documentation/  
├── views.py  
├── drawings.py  
├── sheets.py  
├── dimensions.py  
├── annotations.py  
├── schedules.py  
├── templates.py  
├── renderer.py  
└── package.py
```

85. WORKER PACKAGE

```
apps/worker/  
├── main.py  
├── jobs/  
│   ├── aria.py  
│   ├── geometry.py  
│   ├── validation.py  
│   └── drawings.py
```

```
|
|  ├── documents.py
|  ├── exports.py
|
|  └── runtime/
|      ├── lifecycle.py
|      └── health.py
```

86. API PACKAGE

```
apps/api/
├── main.py
├── routes/
│   ├── projects.py
│   ├── requirements.py
│   ├── model.py
│   ├── aria.py
│   ├── revisions.py
│   ├── redlines.py
│   ├── validation.py
│   ├── drawings.py
│   ├── documents.py
│   ├── jobs.py
│   └── acceptance.py
├── middleware/
├── dependencies/
└── schemas/
```

87. APPLICATION STARTUP

Startup sequence:

```
LOAD CONFIGURATION
↓
INITIALIZE LOGGING
↓
INITIALIZE DATABASE
↓
INITIALIZE QUEUE
↓
INITIALIZE OBJECT STORAGE
↓
INITIALIZE AI ADAPTER
↓
REGISTER ROUTES
↓
START API
```

Workers perform equivalent dependency initialization before accepting jobs.

88. SHUTDOWN

The runtime must support graceful shutdown.

The sequence is:

```
STOP ACCEPTING NEW WORK
↓
FINISH ACTIVE TRANSACTION
↓
ACKNOWLEDGE OR REQUEUE ACTIVE JOB
↓
CLOSE DATABASE
↓
CLOSE QUEUE CONNECTION
↓
CLOSE STORAGE CONNECTION
↓
EXIT
```

89. TESTING STRATEGY

Testing is divided into:

```
unit tests
domain tests
application tests
repository tests
API tests
worker tests
geometry tests
compliance tests
documentation tests
integration tests
end-to-end tests
```

90. DOMAIN TEST REQUIREMENT

Every command must have tests covering:

```
valid execution
invalid execution
authorization failure
constraint conflict
stale revision
missing object
cross-project target
transaction rollback
```

91. WORKER TEST REQUIREMENT

Every asynchronous job must test:

- successful execution
- retry
- failure
- timeout
- stale model version
- duplicate execution
- partial dependency failure

92. ARIA TEST REQUIREMENT

Aria integration tests must verify that natural-language requests are converted into valid structured commands.

Examples:

- move wall
- resize room
- add door
- add window
- change level height
- modify assembly
- create section
- generate drawing set

The resulting command must still pass the normal domain command boundary.

93. CONTRACT TESTING

The OpenAPI specification becomes the API contract.

The implementation must be tested against:

- request schema
- response schema
- error schema
- authentication
- authorization
- status codes
- pagination
- idempotency

94. DATABASE INTEGRATION TEST

The database integration suite shall run against a real PostgreSQL instance rather than only mocks.

The test verifies:

- migration
- foreign keys
- unique constraints
- transactions
- revision creation
- change records
- audit events
- reconstruction

95. END-TO-END RUNTIME TEST

The first runtime-level test is:

```
CREATE PROJECT
↓
CREATE MODEL
↓
ARIA REQUEST
↓
COMMAND VALIDATION
↓
MODEL MUTATION
↓
REVISION CREATION
↓
GEOMETRY JOB
↓
VALIDATION JOB
↓
DRAWING JOB
↓
DOCUMENT JOB
↓
PACKAGE HASH
↓
ACCEPTANCE
```

96. RECONSTRUCTION TEST

Given:

- project_id
- revision_id
- model_version_id

the runtime must retrieve the complete model state required to reconstruct that revision.

The reconstructed state must produce the same:

state hash

where deterministic serialization permits comparison.

97. DETERMINISTIC SERIALIZATION

Model state must have a canonical serialization process.

Conceptually:

```
MODEL OBJECTS
↓
SORT
↓
NORMALIZE
↓
SERIALIZE
↓
HASH
```

This provides stable state fingerprints.

98. MODEL STATE HASH

Every model version contains:

state_hash

The hash provides an integrity reference for the computational project state.

Derived artifacts can then reference the source model hash indirectly through:

model_version_id

99. PROVENANCE CHAIN

The runtime must preserve:

```
USER REQUEST
↓
ARIA INTERPRETATION
↓
COMMAND
```

↓
REVISION
↓
MODEL VERSION
↓
CODE MANIFEST
↓
VALIDATION
↓
DRAWING
↓
DOCUMENT PACKAGE
↓
ACCEPTANCE

This is the OneDraft provenance chain.

100. FAILURE ISOLATION

A failure in:

geometry

must not corrupt:

project model

A failure in:

document generation

must not corrupt:

revision history

A failure in:

AI processing

must not modify:

project state

The runtime must isolate derived processing from authoritative transactional state.

101. MODEL-FIRST PRINCIPLE

The runtime follows:

MODEL FIRST

No generated drawing becomes authoritative simply because it exists.

The computational project model remains the source of truth.

102. IMPLEMENTATION BOUNDARY

The first executable runtime implementation shall contain:

ProjectService
ModelService
CommandService
RevisionService
AriaService
JobService
AuditService

The first worker implementation shall contain:

AriaJob
GeometryJob
ValidationJob
DrawingJob
DocumentJob

103. FIRST RUNTIME ARTIFACTS

The next executable package shall create:

packages/application/services/project_service.py
packages/application/services/command_service.py
packages/application/services/revision_service.py
packages/application/services/aria_service.py
packages/application/unit_of_work.py
packages/infrastructure/database/session.py
packages/infrastructure/queue/client.py
packages/infrastructure/storage/object_store.py
packages/ai/aria/orchestrator.py
apps/worker/main.py
apps/api/main.py

104. FIRST RUNTIME IMPLEMENTATION ORDER

The implementation sequence is:

1. Configuration
2. PostgreSQL session
3. Unit of Work
4. Repository interfaces
5. PostgreSQL repositories
6. Project Service
7. Model Service
8. Command Service
9. Revision Service
10. Audit Service
11. Queue infrastructure
12. Job model
13. Worker runtime
14. Aria adapter
15. Aria orchestration
16. Geometry worker boundary
17. Validation worker boundary
18. Drawing worker boundary
19. Document worker boundary
20. End-to-end integration test

105. DOCKER RUNTIME

Development orchestration shall provide:

api

worker
postgres
redis
object-storage

Conceptually:

`docker-compose.yml`

will expose the complete local OneDraft runtime.

The frontend may run independently during early development.

106. LOCAL DEVELOPMENT FLOW

The developer should eventually be able to execute:

`docker compose up`

and obtain:

PostgreSQL
Redis
Object Storage
OneDraft API
OneDraft Worker

without manually assembling infrastructure.

107. MIGRATION STARTUP RULE

Production application startup must not blindly execute arbitrary destructive migrations.

Database migrations are an explicit deployment operation.

The runtime verifies schema compatibility before accepting traffic.

108. WORKER COMPATIBILITY

Workers must verify that the database schema and application version are compatible before processing jobs.

This prevents a new worker from interpreting old job payloads incorrectly.

109. JOB SCHEMA VERSIONING

Every job payload contains:

`schema_version`

Example:

`geometry.job.v1`
`validation.job.v1`
`drawing.job.v1`
`document.job.v1`

This allows job contracts to evolve safely.

110. EVENT SCHEMA VERSIONING

Events likewise contain:

`schema_version`

This permits downstream consumers to evolve without corrupting historical records.

111. API / APPLICATION / DOMAIN BOUNDARY

The final separation is:

API
↓
APPLICATION
↓
DOMAIN
↓
INFRASTRUCTURE

with specialized execution systems connected through controlled interfaces:

APPLICATION
├── AI
├── GEOMETRY
├── COMPLIANCE
├── DOCUMENTATION
└── JOB SYSTEM

112. NO DIRECT DOMAIN BYPASS

The following architecture is prohibited:

API → PostgreSQL
AI → PostgreSQL
Worker → arbitrary SQL
Frontend → PostgreSQL
Drawing engine → direct model mutation

The required path remains:

REQUEST
↓
APPLICATION SERVICE
↓
DOMAIN COMMAND
↓
UNIT OF WORK
↓
PERSISTENCE

113. RUNTIME SUCCESS CRITERIA

The runtime implementation is successful when it can:

authenticate a request
authorize organization access
load a project
load a model version
execute a domain command
validate constraints
create a revision
record change records
record an audit event
dispatch asynchronous work
execute a worker job
detect stale model versions
store derived artifacts
return job status

maintain provenance

114. ARIA SUCCESS CRITERIA

Aria integration is successful when it can:

- receive project context
- interpret a design instruction
- identify project objects
- produce structured commands
- preserve declared constraints
- submit commands through the domain boundary
- receive validation results
- propose revisions
- explain affected project objects

without directly modifying authoritative persistence.

115. RUNTIME REGRESSION TEST

The complete runtime regression test shall execute:

1. Start PostgreSQL
2. Start Redis
3. Start object storage
4. Start API
5. Start worker
6. Create organization
7. Create user
8. Authenticate
9. Create project
10. Create requirements
11. Create building

12. Create levels
13. Create spaces
14. Create architectural elements
15. Create constraints
16. Submit Aria request
17. Generate structured command
18. Validate command
19. Commit model mutation
20. Create Revision 2
21. Dispatch geometry job
22. Validate geometry
23. Dispatch compliance validation
24. Generate drawings
25. Generate document package
26. Hash artifacts
27. Retrieve job history
28. Retrieve revision history
29. Verify provenance
30. Record acceptance
31. Reconstruct accepted project state

The complete sequence must succeed without direct database access from the client.

116. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now contains:

Product definition

System architecture

Technology architecture

Computational project model

PostgreSQL persistence model

OpenAPI contract

Versioning model

AI command boundary

Regulatory model

Drawing model

Redline model

Validation model

Document provenance

Acceptance workflow

Application service layer

Transaction boundary

Asynchronous job architecture

Worker architecture

AI orchestration boundary

Geometry execution boundary

Compliance execution boundary

Documentation execution boundary

Object-storage boundary

Observability model

Security boundary

Runtime testing model

117. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package shall consist of the runtime foundation:

Artifact A

`packages/application/unit_of_work.py`

Artifact B

packages/application/services/command_service.py

Artifact C

packages/application/services/project_service.py

Artifact D

packages/application/services/revision_service.py

Artifact E

packages/infrastructure/database/session.py

Artifact F

packages/infrastructure/queue/client.py

Artifact G

packages/ai/aria/orchestrator.py

Artifact H

apps/worker/main.py

Artifact I

apps/api/main.py

Artifact J

docker-compose.yml

Together these artifacts establish the executable OneDraft runtime.

118. FINAL ARCHITECTURAL DECISION

OneDraft shall operate as a:

MODEL-CENTRIC
TRANSACTIONAL
EVENT-AWARE
VERSIONED
ASYNC EXECUTION PLATFORM

The API is the external interface.

The application layer is the orchestration layer.

The domain model is the computational authority.

PostgreSQL is the transactional persistence authority.

The worker system performs expensive computation.

Aria interprets and proposes operations.

The geometry engine computes spatial state.

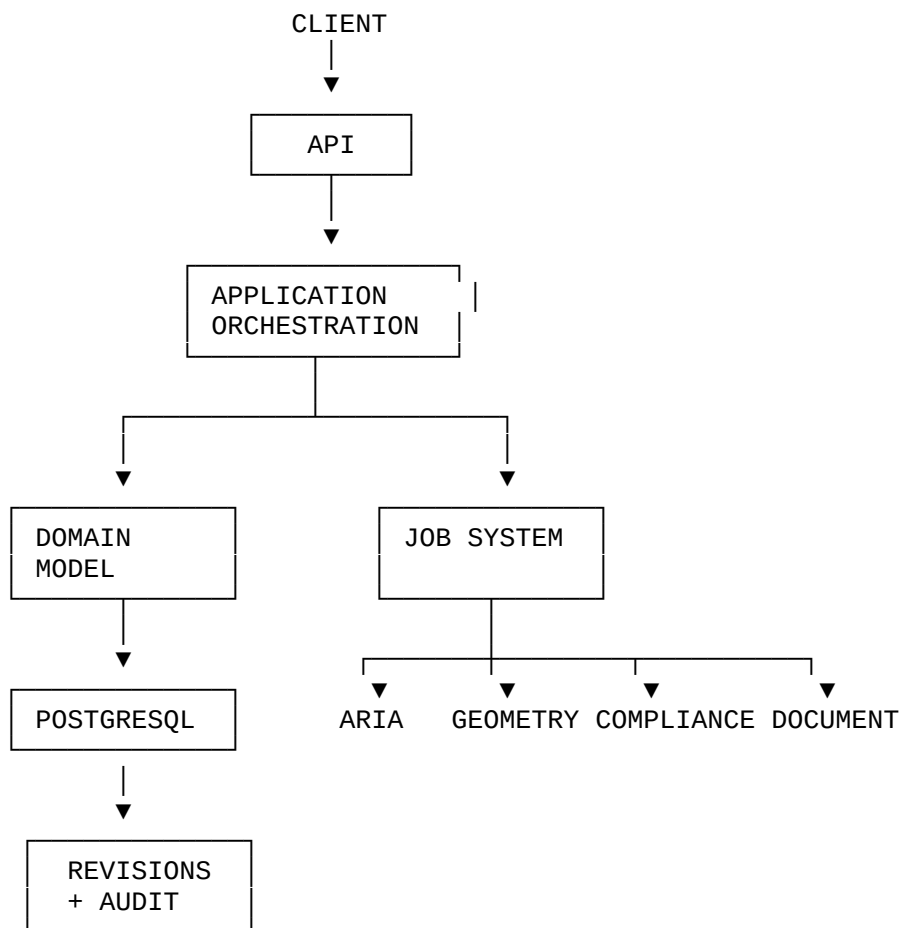
The regulatory engine evaluates applicable requirements.

The documentation engine derives architectural documents.

The audit system preserves the history.

The customer acceptance system establishes the final reviewed package.

The resulting architecture is:



The critical invariant remains:

ARIA DOES NOT OWN THE MODEL.

THE API DOES NOT OWN THE MODEL.

THE DRAWING DOES NOT OWN THE MODEL.

THE PDF DOES NOT OWN THE MODEL.

THE COMPUTATIONAL PROJECT MODEL OWNS PROJECT STATE.

Everything else derives from, operates upon, validates, documents, or records that state.

119. SPECIFICATION STATUS

ONEDRAFT RUNTIME & APPLICATION ORCHESTRATION STACK v0.1

STATUS: ESTABLISHED

The persistence/API architecture is established.

The executable foundation has been defined.

The next implementation stage is the construction of the runtime artifacts beginning with:

`unit_of_work.py`

`command_service.py`

`project_service.py`

`revision_service.py`

`session.py`

`queue/client.py`

`aria/orchestrator.py`

`worker/main.py`

`api/main.py`

`docker-compose.yml`

These artifacts constitute the next actual OneDraft software layer.

120. NEXT STACK POSITION

The OneDraft stack now advances from:

PRODUCT

↓

SYSTEM ARCHITECTURE

↓

DOMAIN MODEL

↓

PERSISTENCE



OPENAPI



APPLICATION ORCHESTRATION



RUNTIME / WORKERS

The next major computational layer after runtime orchestration is:

GEOMETRY ENGINE

+

PARAMETRIC BUILDING MODEL

+

SPATIAL TOPOLOGY

+

DEPENDENCY GRAPH

+

GEOMETRIC CONSTRAINT SOLVER